



Principy programovacích jazyků a OOP

Interpret objektového jazyka SOL26

Interpret: Python, Tester: TypeScript

Dokumentácia

Obsah

[Celkový popis návrhu riešenia](#)

[UML Diagram](#)

[Popis hlavných interných dátových štruktúr](#)

[Zásahy do šablón](#)

[Návrhové vzory a princípy OOP](#)

[Problémy a sporné situácie](#)

[Rozšíriteľnosť](#)

[Využitie AI](#)

[html_generator.ts](#)

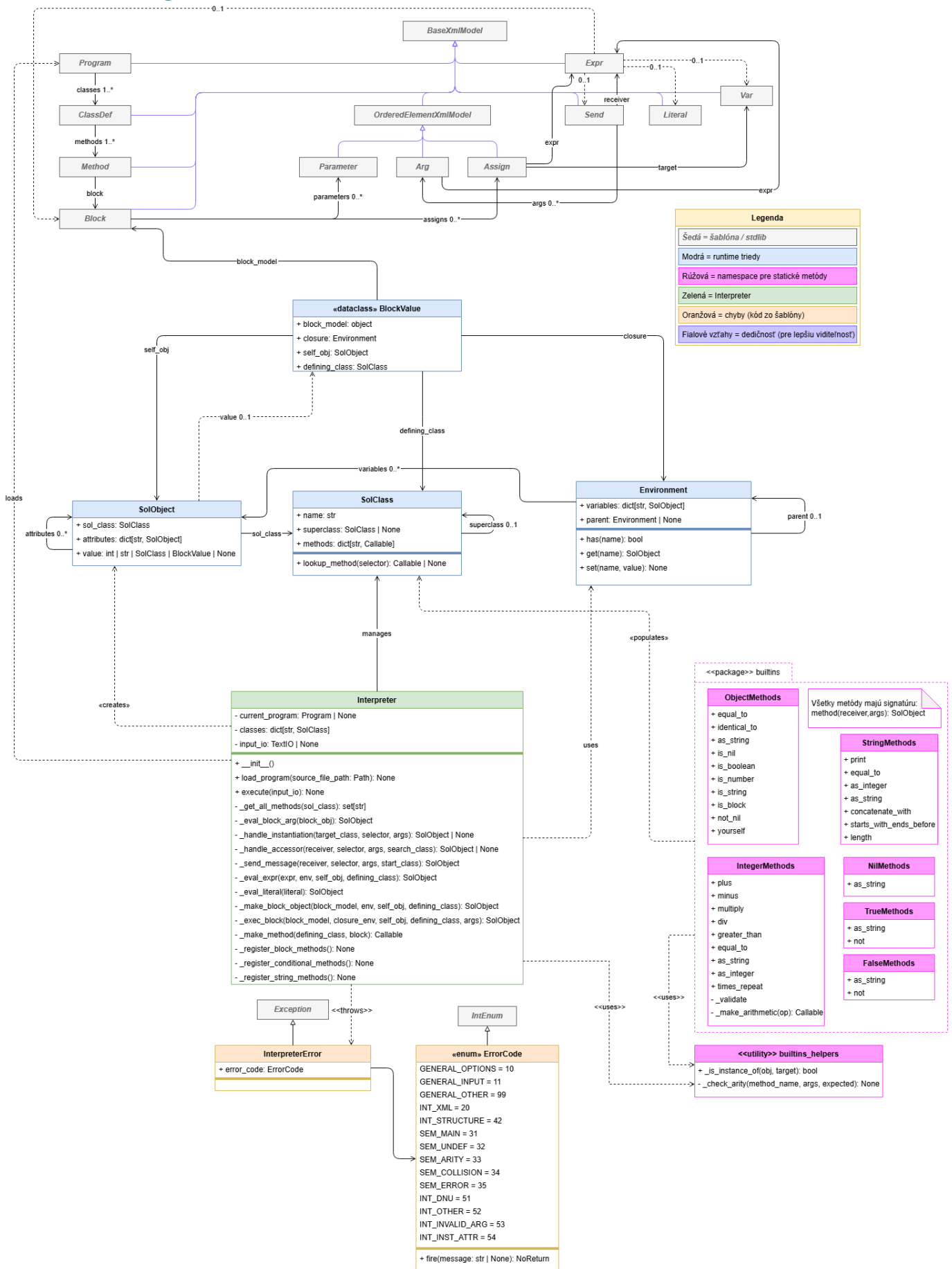
Celkový popis návrhu riešenia

Riešenie projektu pozostáva z dvoch hlavných častí: interpretu jazyka SOL26 napísaného v Pythone a automatizovaného testovacieho rámca napísaného v TypeScript. Oba komponenty sú zabalené do viacstupňového Docker kontajnera, ktorý zabezpečuje reprodukovateľné prostredie pre build, kontrolu kvality kódu aj spúšťanie testov.

Interpreter prijíma SOL-XML súbor, ktorý získame z externého prekladača SOL2XML a vykoná ho. Jadro interpretu tvorí objektový model definovaný v module runtime.py, kde každá hodnota za behu je inštanciou typu *SolObject* nesúca referenciu na svoju triedu, slovník atribútov a prípadnú primitívnu hodnotu. Vstavané triedy jazyka sú definované v module builtins.py. Samotná exekučná logika v interpreter.py načíta XML program, zaregistruje triedy do interného registra a spustí vykonávanie volaním metódy run triedy Main.

Vykonávanie prebieha cez mechanizmus odovzdávania správ (message passing): pre každé volanie metódy interpret vyhľadá selektor v hierarchii tried prijímateľa a vykoná buď vstavanú Python metódu, alebo užívateľsky definovanú metódu zabalenú ako blok s lexikálnym uzáverom (closure). Bloky sú plnohodnotné objekty, čo umožňuje implementovať riadenie toku — podmienky aj cykly — výhradne cez odovzdávanie správ, bez špeciálnych jazykových konštrukcií.

UML Diagram



Popis hlavných interných dátových štruktúr

- `SolObject` – reprezentuje každý objekt v jazyku SOL26. Nesie referenciu na svoju triedu, slovník inštančných atribútov a voliteľnú hodnotu (int, string, block).
- `SolClass` – reprezentácia triedy. Obsahuje meno, referenciu na nadtriedu a slovník metód indexovaných selektorom. Na hľadanie metód slúži metóda `lookup_method`, ktorá prechádza hierarchiu metód až ku koreňu pričom hľadá metódu so zadaným selektorom.
- `BlockValue` – uzáver kombinujúci AST uzol bloku s jeho zachyteným lexikálnym prostredím, odkazom na self a definujúcou triedou. Používa sa pre všetky bloky.

Zásahy do šablóny

Pri implementácii interpreta neboli zmenené žiadne šablóny.

Šablóna bola zmenená len v jednom module testeru – `tester.ts`. Bolo to z dôvodu vytvorenia modulu `html_generator.ts` (nebol v súčasťou zadania) pri testeri. Tento zásah nijako nezmenil pôvodné fungovanie šablóny. Všetky zmeny šablóny som okomentoval s mojim loginom na začiatku komentáru (`// xgaspim00: ...`).

Zásahy:

- Pridanie importov
- Zverejnenie deklarácie rozhrania (`export interface CliArguments {...}`)
- Pridanie generovania HTML do function `writeResult(...)`
- Pridanie HTML parametra do informačnej správy
- Pridanie možnosti generovania HTML do `PARSE_OPTIONS`
- Pridanie HTML flagu do `CliArguments`

Ohľadom modulu `html_generator.ts` sa dozviete viac [tu](#).

Návrhové vzory a princípy OOP

- **Stratégia (cez funkcie prvej triedy)** — Všetky metódy tried (vstavané aj užívateľské) sú uložené v slovníku `SolClass.methods` ako volateľné objekty s jednotným podpisom `(receiver, args) -> SolObject`. Metóda `_make_method` v interprete obalí užívateľsky definovaný blok do rovnakého tvaru. Odosielanie v `_send_message` tak nepotrebuje rozlišovať, či volá vstavanú alebo užívateľskú metódu — pracuje vždy s rovnakým rozhraním.
- **Továrenská metóda (Factory Method)** — Metóda `_make_arithmetic` v triede `IntegerMethods` slúži ako továreň, ktorá pre daný operátor (+, -, *, /) vráti konkrétnu volateľnú implementáciu aritmetickej metódy. Vďaka tomu sa štyri podobné metódy neduplikujú, ale generujú z jedného vzoru, čo uľahčuje prípadné rozšírenie o ďalšie operátory.
- **Oddelenie zodpovedností (Separation of Concerns)** — Vstavané metódy nezávislé od behu interpreta sú oddelené do `builtins.py`, metódy vyžadujúce prístup k runtime kontextu (čítanie vstupu, vykonávanie blokov v cykloch a podmienkach) sú injektované priamo interpretom v `_register_*` metódach. Toto rozdelenie zabraňuje kruhovým závislostiam a jasne oddeľuje statickú a dynamickú časť vstavaných tried.
- **Kompozit (rekurzívna stromová štruktúra)** — AST program je reprezentovaný ako vnorená hierarchia Pydantic-XML modelov (`Program` → `ClassDef` → `Method` → `Block` → `Assign` → `Expr` → `Send/Literal/Var/Block`). Každý uzol stromu sa vyhodnotí rekurzívnym volaním `_eval_expr`, čo zodpovedá vzoru Kompozit — zložené výrazy (napr. vnorené volania správ) sa spracovávajú rovnakým mechanizmom ako jednoduché literály.

Problémy a sporné situácie

- Chvíľku som bojoval so situáciou kedy v šablóne `input_model.py` `Arg` referencoval `Expr` pred jej definíciou. Toto som vyriešil pomerne rýchlo tým, že som nainštaloval správnu verziu pythonu. Pôvodne som mal Python 3.12, v ktorom to spôsobovalo `NameError`.
- Rozdelovanie metód do viacerých metód kvôli nástroju Ruff – počas projektu sa mi viackrát stalo, že som narazil na problémy s nástrojom Ruff, ktorý mi hlásil príliš vysokú komplexitu. Toto som riešil jednoducho tak, že som metódy rozdelil do viacerých metód a volal pomocné metódy z tej originálnej.
- Pôvodne som na výpis používal `sys.stdout.write` ale neskôr som si uvedomil že stačí použiť `print(x, end = "")`
- Narážal som na chybu `SEM_UNDEF` pri kľúčovom slove `super`. Detekoval som ho správne, no chyba sa stále objavovala. Neskôr som zistil, že nemám podmienku v metóde `_eval_expr`.

Rozšíriteľnosť

Mechanizmus `doesNotUnderstand (DNU)` – Toto rozšírenie by si vyžiadalo minimálne zásahy do existujúceho kódu. V metóde `_send_message` už existuje miesto, kde sa vyhodí chyba `INT_DNU` — práve tam by stačilo pred vyhodnotením chyby overiť, či príjemca (alebo niektorý predok v hierarchii) definuje metódu `doesNotUnderstand`. Ak áno, zavolá sa s novým objektom `Message`, ktorý zapuzdruje selektor a argumenty. Objekt `Message` by bol nová vstavaná trieda registrovaná v `builtins.py` rovnakým spôsobom ako existujúce triedy. Počiatočnú implementáciu `doesNotUnderstand` by stačilo pridať do `OBJECT_CLASS.methods`. Ochrana proti rekurzii by sa dala riešiť jednoduchým setom navštívených (`receiver_id`, `selector`) dvojíc v rámci jedného volania.

Triedy ako objekty prvej kategórie – Základ pre toto rozšírenie je v interprete už čiastočne prítomný — triedy sú v SOL26 reprezentované ako `SolObject` s hodnotou `value: SolClass`, čo znamená, že objekt triedy už existuje ako bežný runtime objekt. Rozšírenie by si vyžiadalo pridanie novej vstavanej triedy `Class` do `builtins.py` a registráciu správ `name`, `superclass`, `methods` a `instanceOf`: do jej slovníka metód — rovnakým mechanizmom ako sú registrované metódy `Integer` či `String`. Správa `class` na každom objekte by sa pridala do `OBJECT_CLASS.methods` ako jednoduchá metóda vracajúca `SolObject` obaľujúci `sol_class` príjemcu. Podpora triednych atribútov a triednych metód by si vyžiadala rozšírenie `SolClass` o druhý slovník metód (`class_methods`) a slovník zdieľaných atribútov — `_send_message` by pri príjemcovi typu `SolClass` prehľadávalo tento slovník pred inštančnými metódami, čo je miesto, kde sa táto logika už čiastočne nachádza.

Využitie AI

Na projekt som využíval výhradne nástroj Github Copilot s agentom Gemini 3.1 Pro (Preview).

Všetky konverzácie sú priložené v dokumente ai_github_copilot.pdf

Použitie:

- Cyklické závislosti – pri implementácii vznikol problém s cyklickou závislosťou medzi triedami `Environment` a `SolObject`. Nástroj analyzoval problém a ponúkol mi 3 riešenia. Vybral som si konsolidáciu. Presunul som triedy do `runtime.py`. Tým sa eliminovala cyklická závislosť
- Trieda `Environment` – v tomto prípade som si len overoval moje navrhované riešenie, pretože som mal ohľadom toho rôzne pochybnosti. Času už nebolo veľa, preto som sa rozhodol to skonzultovať s Agentom.
- TypeScript – s týmto programovacím jazykom som nerobil mnohokrát a preto som potreboval informácie ohľadom syntaxe a knižníc, ktoré by som mal použiť. Neskôr som robil už vlastný prieskum knižníc, pretože som narážal na problémy s informáciami, ktoré mi poskytol Agent.
- Dockerfile – tento typ súboru som predtým ešte netvoril, preto som potreboval trochu naštartovať. Vzhľadom na časovú tieseň a neznalosť tohto typu, som si nechal vytvoriť prvý Stage v Dockerfile. S týmto návrhom a následným dohľadom pár informácií som pochopil čo mám robiť a zvyšok súboru už som zvládol sám.
- Kontrola komentárov – konzultoval som s Agentom moje komentáre a nechal som ho automaticky opraviť moje preklepy. Zhodli sme sa že v kóde mám veľa nepotrebných komentárov. Kód bol čitateľný a zrozumiteľný aj bez týchto komentárov a preto som skoro polovinu komentárov odstránil. Agent odhalil na druhej strane aj miesta, kde som komentár nepridal a hodil by sa tam. Toto odstránenie a pranie komentárov som robil manuálne.
- Tvorba testov – pár testov som si vytvoril sám. Potom som spoločne s Agentom vytvoril ďalšie testy na overenie funkčnosti môjho projektu. Využil som k tomu špecifikáciu jazyka SOL26. Testy neboli dokonalé a mnoho z nich som musel ručne opraviť.
- [html_generator.ts](#)

html_generator.ts

Tento modul nebol súčasťou zadania. Bol vygenerovaný umelou inteligenciou. Nepáčilo sa mi hľadať výsledky v JSON reporte, preto som poveril Agentu s úlohou vytvoriť generovanie HTML reportu na základe reportu JSON.

Týmto som si chcel vyskúšať aké ťažké by bolo vygenerovať takýto modul pre projekt. Na moje počudovanie stačilo Agentovi pár promptov.

Nebolo to hneď dokonalé ale po pár úpravách a dodatočných promptoch som priviedol tento modul do stavu, s ktorým som spokojný.

Modul sa aktivuje flagom --html v kombinácii s -o <súbor>, pričom HTML report sa vygeneruje s rovnakým názvom ako JSON výstup.

Ukážka vygenerovaného HTML:

